Hands-on Activity 10.2 | Exploring Core Components and Component Styling	
Course Code: CPE 025B	**Program: BSCPE**
Course Title: Software Design 2	**Date Performed: 03/27/26**
Section: CPE32S1	**Date Submitted: 03/27/26**
Name: Edson Ray San Juan	**Instructor:** Engr. Neil Matira

1. Objective(s)

This activity aims to create a basic react native application to enable students to learn how to use core components and style them.

2. Intended Learning Outcome (ILO):

After this module, the student should be able to:

- Utilize React Native components and build user interfaces;
- Style React Native applications; and
- Implement additional interactivity and manage states.

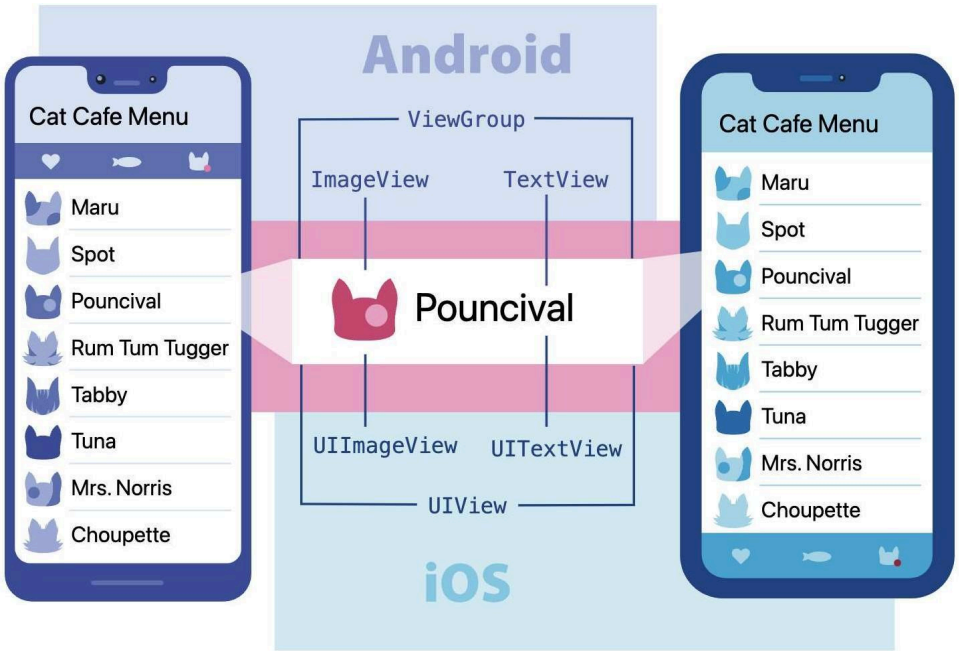
3. Discussion

Core Components and Native Components

React Native is an open source framework for building Android and iOS applications using React and the app platform's native capabilities. With React Native, you use JavaScript to access your platform's APIs as well as to describe the appearance and behavior of your UI using React components: bundles of reusable, nestable code.

Views and mobile development

In Android and iOS development, a view is the basic building block of a UI: a small rectangular element on the screen which can be used to display text, images, or respond to user input. Even the smallest visual elements of an app, like a line of text or a button, are kinds of views. Some kinds of views can contain other views. It's views all the way down!



Native Components

In Android development, you write views in Kotlin or Java; in iOS development, you use Swift or Objective-C. With React

Native, you can invoke these views with JavaScript using React components. At runtime, React Native creates the corresponding Android and iOS views for those components. Because React Native components are backed by the same views as Android and iOS, React Native apps look, feel, and perform like any other apps. We call these platform-backed components Native Components.

React Native comes with a set of essential, ready-to-use Native Components you can use to start building your app today. These are React Native's Core Components.

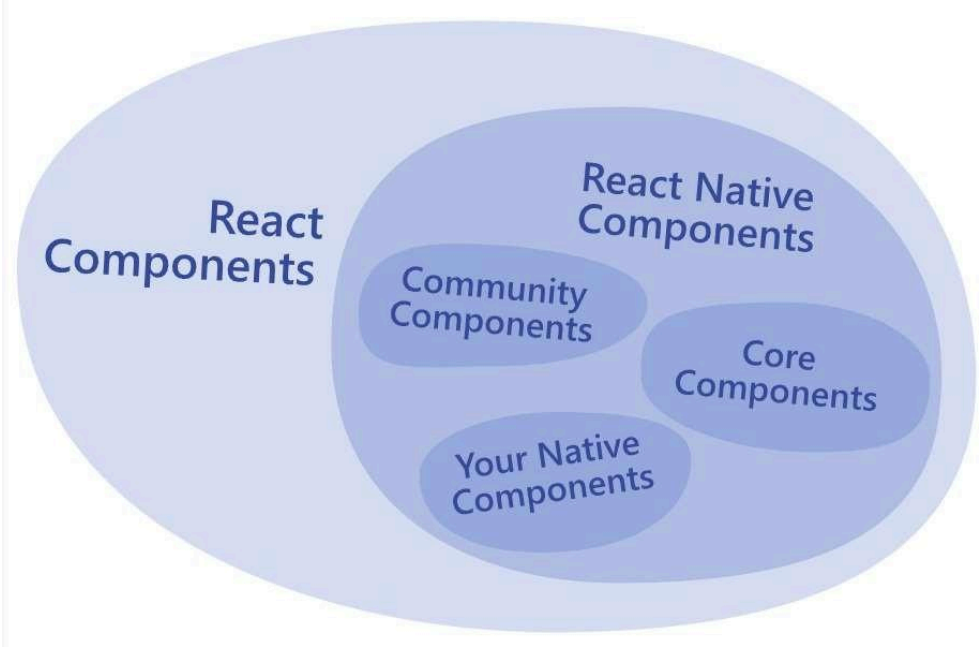
React Native also lets you build your own Native Components for Android and iOS to suit your app's unique needs. We also have a thriving ecosystem of these community-contributed components. Check out Native Directory to find what the community has been creating.

Core Components

React Native has many Core Components for everything from controls to activity indicators. You can find them all documented in the API section. You will mostly work with the following Core Components:

REACT NATIVE UI COMPONENT	ANDROID VIEW	iOS VIEW	WEB ANALOG	DESCRIPTION
<View>	<ViewGroup>	<UIView>	A non scrolling	A container that supports layout with flexbox, style, some touch handling, and accessibility controls
<Text>	<TextView>	<UITextView>	<p>	Displays, styles and nests strings of text and even handles touch events.
<Image>	<ImageView>	<UIImageView>	<img>	Displays different types of images.
<ScrollView>	<ScrollView>	<UIScrollView>		A generic scrolling container that can contain multiple components and views.
<TextInput>	<EditText>	<UITextField>	<input type="text">	Allows the user to enter text

Because React Native uses the same API structure as React components, you'll need to understand React component APIs to get started. The next section makes for a quick introduction or refresher on the topic. However, if you're already familiar with React, feel free to skip ahead.



Discussion content taken from official React Native documentation. View more here https://reactnative.dev/docs/intro-react-native-components and read more.

4. Materials and Equipment

To properly perform this activity, the students must have:

- Node.js LTS 18.7.1
- Visual Studio Code
- Android Emulator

5. Procedure

Before starting with the procedures, ensure that you have the project from the previous activity. If you have not, you may recreate the project and install dependencies such as npm install. You may then check by running the commands a or i to run on your selected Android or iOS devices, respectfully. Note: You may also choose to preview the app on the web by pressing w. This will require you to install dependencies but is not recommended for the class.

By exploring the previously created project, your app.js may look similar to the figure. The figure below shows what is call a React component, specifically a functional React component.

```
export default function App() {
  return (
    <View style={styles.container}>
      <Text>Team Name</Text>
      <Text>Roman Richard</Text>
      <StatusBar style="auto" />
    </View>
  );
}
```

Aside from this, you can find a styles constant - the StyleSheet object - below.

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

At the top, there are imports to import one component from an Expo related package, and other components from React Native.

```
JS App.js > ...
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View } from 'react-native';
```

Text and View are two of some built-in components that React Native exposes to you to use in your JSX code, within the functional component, you cannot use h2 tags or any other HTML elements that work in a browser.

Setting Up Your Android Simulator

Option 1: Using Android Studio Simulator.

1. Start Expo by typing expo start on the command line.
2. Open Android Studio.
3. Open your project in Android Studio.
4. Open the 'AVD manager' from the Android Studio toolbar.
5. Run an Android emulator.
6. Click 'run on Android device/emulator' in the Expo client.

Option 2: Using any Android Emulator. This will not include any steps; all it entails are the same things you've done on your actual device. Copy and paste the socket from your terminal to the expo go app to run your application demo.

Checking Out Simple Core Components

1. In your app.js delete the import for the expo-status-bar and delete StatusBar under View.
2. Save the app.js and see if there are any changes to the output.
3. Delete the <Text></Text> component tags in your code. Your code should look similar to below:

```
import { StyleSheet, Text, View, Button } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      Team Name
      Roman Richard
    </View>
  );
}
```

4. Save app.js. Screenshot your output. What does this mean? Make sure that for this step, you are previewing the app on your phone and not web deployment.
5. Preview it on the web. What output do you get?

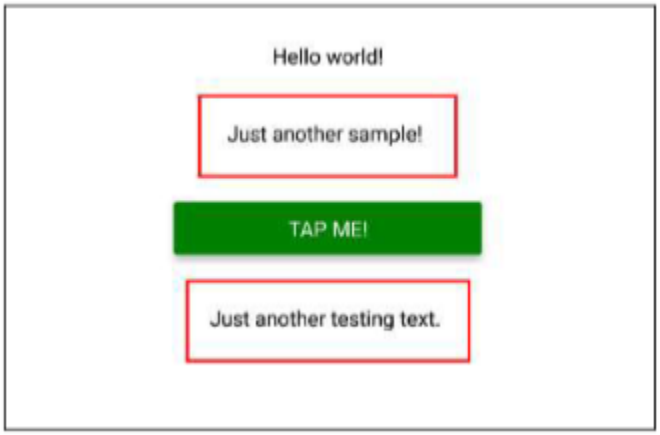
6. Add a nested View in your first View component and add a button by adding <Button title = "Tap me!" />.
7. Your output must be similar to this figure below. (Disregard the kind of simulator used in the figure)

Styling React Native Apps

1. Create a new Text component with the name of your team members.
2. For each Text component, add a margin of 16. Show the output.
3. Next, add a border: try a border through a provided shortcut like 'blue' and through hex code. Show the output for both.
4. Next, add a padding with the same value as the margin. Show the output.
5. Add a 'color' style to the Button component with the value "green". Show the output.

We will further explore additional styling methods as we go further. However, feel free to check the React Native documentation if you wish to explore more. If you only followed the steps above, your app should look similar to the figure below.

6. Create another View component with a dummy text nested alongside the component we added with a red border. Copy the style from the old Text in the View component to the new one. Your output must be similar below.

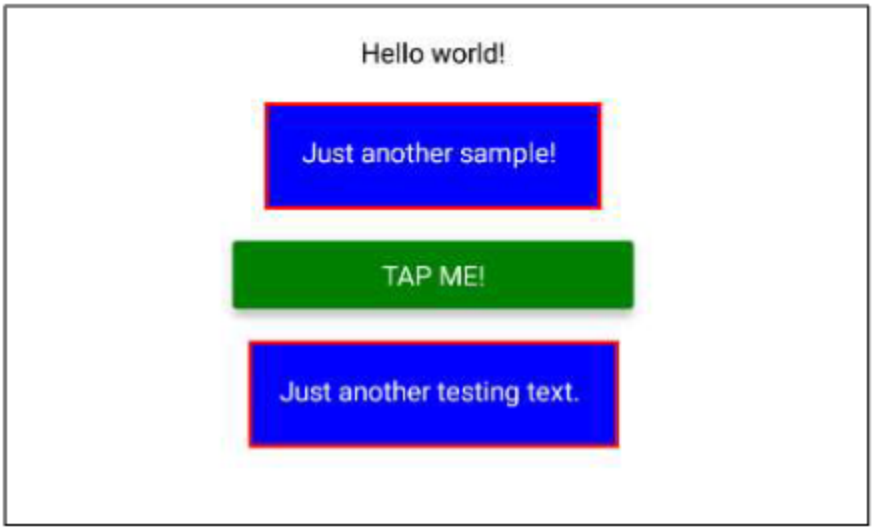


7. However, this is inefficient to have to do repeatedly. Instead, we will create an object under our StyleSheet. Your code must now be similar to the figures below. Did your output change? Why/Why not?

```
export default function App() {
  return (
    <View style={styles.container}>
      <Text>Hello world!</Text>
      <View>
        <Text style={styles.textStyle}>
        <Button title="Tap me!" color="
      </View>
      <View>
        <Text style={styles.textStyle}>
      </View>
    </View>
  );
```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  textStyle: {
    margin: 16,
    borderWidth: 2,
    borderColor: 'red',
    padding: 16,
  }
```

8. Add a new style to the textStyle StyleSheet component. Background color should be 'blue' and the color should be 'white'. Save the file and observe the app, what changed? Was this method more convenient? Your output must be similar to the figure shown.



9. Create a new style property called Styles2, this will not use the imported React Native StyleSheet nor its built-in methods. The figure below shows the steps taken and the corresponding output. Show your output.

```
const styles2 = {

};
```

```
const styles2 = {
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  textStyle: {
    margin: 16,
    borderWidth: 2,
    borderColor: 'red',
    backgroundColor: 'blue',
    color: 'white',
    padding: 16,
  }
};
```

```
export default function App() {
  return (
    <View style={styles.container}>
      <Text>Hello world!</Text>
      <View>
        <Text style={styles2.textStyle}
        <Button title="Tap me!" color="
      </View>
      <View>
        <Text style={styles2.textStyle}
      </View>
    </View>
  );
}
```

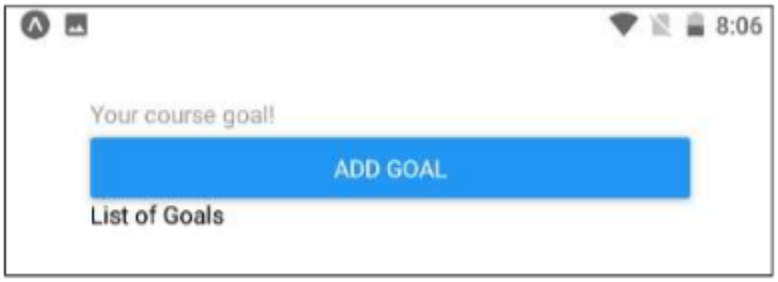
FOR THE NEXT SECTION: We will start creating a basic app that will help us manage our goals. To do this, we will first clean up our JSX. Before continuing, your code should look similar to what is shown below:

```
JS App.js > ...
1  import { StyleSheet, Text, View, Button } from 'react-native';
2
3  export default function App() {
4    return (
5      <View>
6        <View></View>
7        <View></View>
8      </View>
9    );
10 }
11
12 const styles = StyleSheet.create({
13
14 });
```

Exploring Layouts and Flexbox

To create our goal manager app, perform the following:

1. In the first nested View, add a TextInput component (don't forget to import!) with a placeholder containing the string 'Your Course Goal'.
2. Next, add a Button below it with the title 'Add Goal'.
3. In the second nested View, add a Text component that says "List of Goals". This will be a goal later on.
4. Show the output.
5. Create a styles object called appContainer meant to contain the entire app with a padding of 50 pixels. Your output must be similar to the output below.



Now, we will use FlexBox

1. Our goal is to make the button and the textinput components be next to each other. To do this, we are going to create a new styles object for the view that holds both components-we will call it inputContainer. Add flexDirection set to row and justifyContent set to space-between. Assign it to that View component.
2. Show your output from #1.
3. Next, we want to distinguish the textInput component. Create a new object under styles called textinput, add a border with a width of 2 and a color of #cccccc. Assign to the TextInput then show the output.
4. Next, add a width of 80%. Make sure it is a string. Show the output.
5. Add a marginRight with a value 8 and a padding value 3. Show the output.
6. Your output must be similar to the figure below.

Improving the Layout

1. Change paddings in appContainer to padding Top: 50, padding Horizontal: 16 and adding textInput width to 70% and padding to 13. Show the output so far.
2. Notice the weird placement of the text on your button now. Try to add a style to prop to fix this. What happened?
3. Add alignitems: center to the inputContainer. Did this fix the problem? What does this do?
4. To further fix the layout, add a paddingBottom to the InputContainer with a value of 24, create a 1 pixel border on the bottom with the same color as the textInput's border. Show the result.
5. The textInput area has to take ¼ of the overall available height. To do this, a flex property has to be added. Add flex 1 to the inputContainer.
6. We are also going to add a new object under styles for the second nested view to support this. Create a new object called goalsContainer and give it a flex value of 3.
7. Show the output so far. Is that the expected output?
8. Working on the inner containers isn't enough. The outer container must also be adjusted. The appContainer must be forced to take all the available height. Add flex: 1. Show the output.
9. Change the flex in goalsContainer to 4. Change paddingBottom to marginBottom in the inputContainer. Show the output.
10. Change the flex in goalsContainer to 5. Show the output. This will be your final layout for now as we proceed with the next steps.

Handling Events

Now, we have a textbox for user input and a button that should place this text into the goal list given when the button is tapped. In order to make these happen, we have to handle events.

1. Make two functions and add them before the return statement in your export default function App():

function goalInputHandler() ();
function addGoalHandler() {};

It must now look similar to the figure below.

```
export default function App() {
  function goalInputHandler() {};

  function addGoalHandler() {};

  return (
```

2. First, we will connect the textinput component that should take in the texts given by the user through the use of the onChange Text={} prop built-in with React Native. The onChange Text prop wants a function as a value; a pointer at a function. So, we'll pass the goalInputHandler. But not as a function call.

This now means that the goallinputHandler function receives the entered value automatically. The entered value becomes the parameter to the function, so we add a parameter to this function called entered Text.

To check, add console.log(entered Text); to the body of the function. Save and then type "Testing" to see whether the connection works properly.

Your output should be similar to the figure shown below.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL

LOG  T
LOG  Te
LOG  Tes
LOG  Test
LOG  Testi
LOG  Testin
LOG  Testing
```

3. However, this saved text should be handled by the button, not the text box. So, now that we've confirmed that the value per keystroke is passed, remove the contents of goalInputHandler's function body.

In React Native, we have to use the onPress={ prop to connect our function passed as input to the prop. It should now appear as onPress={addGoalHandler};, and similar to earlier, it must not be in the form of a function call.

4. Now that it's connected, we want to get the enteredText in the textInput to be handled by our button. We can do this through states. At the top of your imports, include:

import {useState} from 'react';

To use the state, first call useState and set as an empty string initially.

const[]= useState('');

The idea is we want to include within the the user input state or enteredGoalText state which can be updated by the setEnteredGoalText function. This is standard React syntax.

5. Going back to goalInputHandler, we want to call the update function from the useState call above and pass the entered Text as its parameter. Your code should look similar to the figure below.

```
export default function App() {
  const [enteredGoalText, setEnteredGoalText] = useState('');

  function goalInputHandler(enteredText) {
    setEnteredGoalText(enteredText);
  };

  function addGoalHandler() {};
```

6. We now want to test if we have access to this text from the useState. Add console.log(enteredGoalText); in your addGoalHandler's function body. Save and type down "Testing and your output should be similar to the figure below.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL

LOG  Testing
```

By the end of step 6, your code should be similar to the figure below.

```
JS App.js > ...
1  import { useState } from 'react';
2  import { StyleSheet, Text, View, Button, TextInput } from 'react-native';
3
4  export default function App() {
5    const [enteredGoalText, setEnteredGoalText] = useState('');
6
7    function goalInputHandler(enteredText) {
8      setEnteredGoalText(enteredText);
9    };
10
11   function addGoalHandler() {
12     console.log(enteredGoalText);
13   };
```

Managing the List of Course Goals

Now, we want to be able to update our list of goals that found in the secondary view component.

1. Typically, when data changes dynamically we use state. Such as in this case, with the data for the list of goals changing, we want to useState as well. Add another useState below the first one created in the earlier steps. However, this second state will be initialized with an empty array instead.

   This state should be named courseGoals that can be changed by the setCourse Goals function.

2. In the body of your addGoalHandler, instead of using the console, use the setCourseGoals. We want to be able to take your existing goals and append (add to the end) a new one.

   We do this by passing an array to the setCourseGoals function.
   setCourse Goals([]);

   Then we use the spread operator to spread existing course goals into the new array, keeping all the existing goals.

   setCourse Goals([...courseGoals]);

   And then adding the new goal.

   setCourse Goals([...courseGoals, enteredGoal Text]);

   HOWEVER, this is also not the best way to update a state if the new state depends on the previous state. Instead of the aforementioned code, we will pass a function to the state updating function that will be automatically called by react.

   We will use an arrow function instead.

   setCourse Goals((currentCourseGoals) => [...currentCourseGoals, enteredGoal Text,]);

3. At this point, your code should look similar to the figure below.

```
JS App.js > App
import { useState } from 'react';
import { StyleSheet, Text, View, Button, TextInput } from 'react-native';

export default function App() {
  const [enteredGoalText, setEnteredGoalText] = useState('');
  const [courseGoals, setCourseGoals] = useState([]);

  function goalInputHandler(enteredText) {
    setEnteredGoalText(enteredText);
  };

  function addGoalHandler() {
    setCourseGoals((currentCourseGoals) =>[
      ...currentCourseGoals,
      enteredGoalText,
    ]);
  };

  return (
```

4. The next goal is to output the list of goals. Since this data is dynamic, we can now refer to courseGoals enclosed in curly brackets then call the map method on it.

{currentGoals.map()}

We will pass here another arrow function, which takes individual courseGoals for the parameter and as a return, it returns a JSX element.

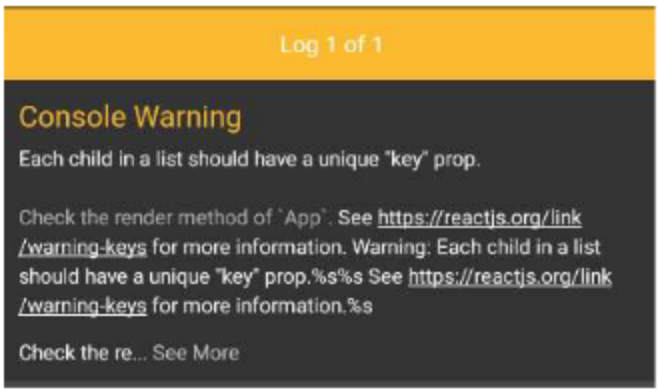
(currentGoals.map((goal) => <Text>{goal}</Text>)}

At this point, your code should look like the figure below.

```
  return (
    <View style={styles.appContainer}>
      <View style={styles.inputContainer}>
        <TextInput
        placeholder='Your course goal!'
        style={styles.textInput}
        onChangeText={goalInputHandler}
        />
        <Button title='Add Goal' onPress={addGoalHandler} />
      </View>
      <View style={styles.goalsContainer}>
        {courseGoals.map((goal) => <Text>{goal}</Text>)}
      </View>
    </View>
  );
}
```

5. Save and run your code, Input in the textInput area as a goal "Leam React Native". Press on "Add Goal" button. What happens? Was it added? Did anything else happen? Show output/screenshots.

You should receive a warning. Not because of React Native, but due to the nature of React in general. The error states that when outputting a list of data every item should receive a key prop.



To make it quick, this is meant for React to manage the list in a more efficient way. Always add the key prop when dealing with individual items being outputted as part of a list.

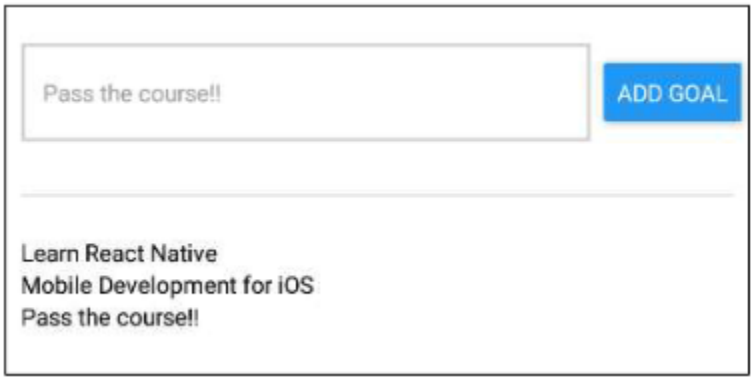
6. The value passed to the key **can be anything** but it must be unique. It has to uniquely identify the concrete value that's being outputted. In this case, each string is assumed to be unique so we'll use the goal string itself.

   <Text key={goal}>{goal}</Text>

   Temporarily, it will work. Test it with different goals and same goals. Such as:

   "Learn React Native" / "Mobile Development for iOS" / "Pass the Course" (Unless you do not want to learn anything and not pass the course.)

The app in your simulator should look similar to the figure below.



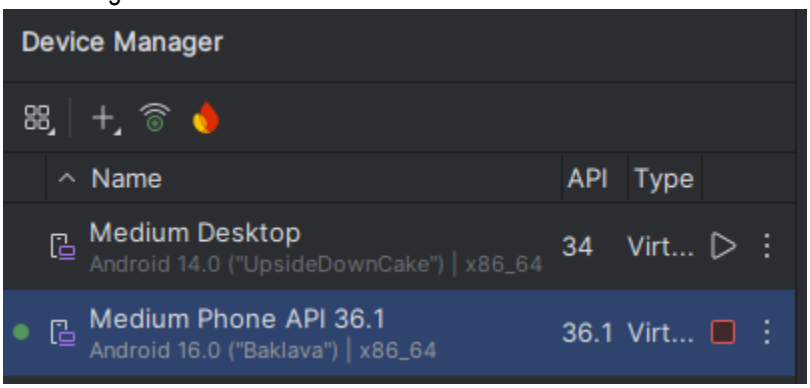
Make sure to save your progress! The application will be continued/finished in the next activity.

6. Output

Setting Up Your Android Simulator

Option 1: Using Android Studio Simulator.

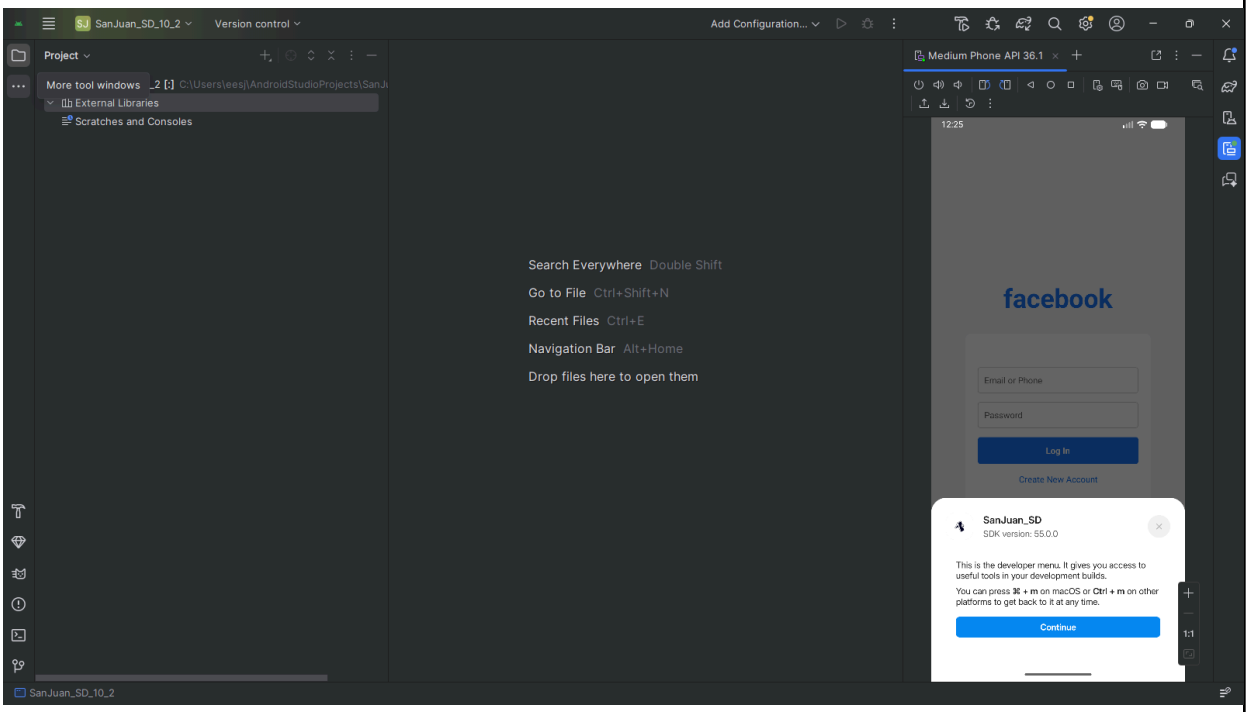
1. Start Expo by typing expo start on the command line.
2. Open Android Studio.
3. Open your project in Android Studio.
4. Open the 'AVD manager' from the Android Studio toolbar.




```
C:\Users\eesj>adb devices
List of devices attached
emulator-5554   device
```

Checking if there are devices listed

5. Run an Android emulator.



6. Click 'run on Android device/emulator' in the Expo client.

- npx expo start

```
Logs for your project will appear below. Press Ctrl+C to exit.
› Opening on Android...
› Opening exp://192.168.254.101:8081 on Medium_Phone_API_36.1
Downloading the Expo Go app [========================================                    ] 65% 8.3s
```

Styling React Native Apps

1. Create a new Text component with the name of your team members.

```
<Text style={styles.memberBlue}>Edson Ray San Juan</Text>
```

2. For each Text component, add a margin of 16. Show the output.

```
memberBlue: {
  fontSize: 20,
  margin: 16,
  borderWidth: 2,
  borderColor: 'blue',
  padding: 16,
},
```

3. Next, add a border: try a border through a provided shortcut like 'blue' and through hex code. Show the output for both.

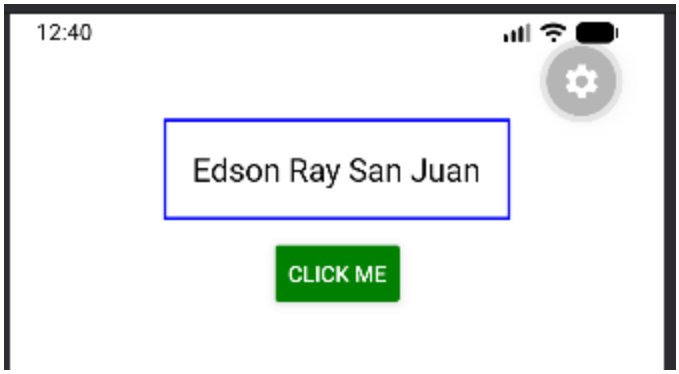
```
memberHex: {
  fontSize: 20,
  margin: 16,
  borderWidth: 2,
  borderColor: '#0000FF',
  padding: 16,
},
});
```

4. Next, add a padding with the same value as the margin. Show the output.

```
memberHex: {
  fontSize: 20,
  margin: 16,
  borderWidth: 2,
  borderColor: '#0000FF',
  padding: 16,
},
});
```

5. Add a 'color' style to the Button component with the value "green". Show the output.

```
  {/* Button with green color */}
  <Button title="Click Me" color="green" onPress={() => {}} />
</View>
```



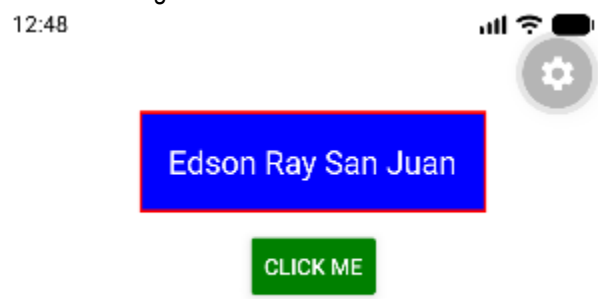
JavaScript

```javascript
import React from 'react';
import { Text, View, Button, StyleSheet } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      {/* Team member names */}
      <Text style={styles.memberBlue}>Edson Ray San Juan</Text>

      {/* Button with green color */}
      <Button title="Click Me" color="green" onPress={() => {}} />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    marginTop: 50,
    alignItems: 'center',
  },
  // Using shortcut color name
  memberBlue: {
    fontSize: 20,
    margin: 16,
    borderWidth: 2,
    borderColor: 'blue',
    padding: 16,
  },
  // Using hex code for border
  memberHex: {
    fontSize: 20,
    margin: 16,
    borderWidth: 2,
    borderColor: '#0000FF',
    padding: 16,
  },
});
```

7. However, this is inefficient to have to do repeatedly. Instead, we will create an object under our StyleSheet. Your code must now be similar to the figures below. Did your output change? Why/Why not?

- **Did your output change?**
  - No, the output did not change because the style rules applied were exactly the same.
- **Why/Why not?**
  - Why? Because the difference is only in code organization, since using a StyleSheet centralizes styles for efficiency without altering the visual result.

8. Add a new style to the textStyle StyleSheet component. Background color should be 'blue' and the color should be 'white'. Save the file and observe the app, what changed? Was this method more convenient? Your output must be similar to the figure shown.



- What changed?
  - Having the rectangle fill filled with blur color backgroundColor: 'blue', and having the border red **borderColor: 'red'.**
- Was this method more convenient?
  - This method was more convenient since using the StyleSheet allowed you to apply and reuse styles in one place without repeating code.

JavaScript

```javascript
import React from 'react';
import { Text, View, Button, StyleSheet } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      {/* Team member name */}
      <Text style={styles.textStyle}>Edson Ray San Juan</Text>

      {/* Button with green color */}
      <Button title="Click Me" color="green" onPress={() => {}} />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    marginTop: 50,
```

```
    alignItems: 'center',
  },
  textStyle: {
    fontSize: 20,
    margin: 16,
    borderWidth: 2,
    borderColor: 'red',
    padding: 16,
    backgroundColor: 'blue',
    color: 'white',
  },
});
```

9. Create a new style property called Styles2, this will not use the imported React Native StyleSheet nor its built-in methods. The figure below shows the steps taken and the corresponding output. Show your output.

JavaScript

```
import React from 'react';
import { Text, View, Button } from 'react-native';

const styles2 = {
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  textStyle: {
    margin: 16,
    borderWidth: 2,
    borderColor: 'red',
    backgroundColor: 'blue',
    color: 'white',
    padding: 16,
```

```
  },
};

export default function App() {
  return (
    <View style={styles2.container}>
      <Text>Hello world! Just another sample!</Text>

      <View>
        <Text style={styles2.textStyle}>
          <Button title="Tap me!" color="green" />
        </Text>
      </View>

      <View>
        <Text style={styles2.textStyle}>
          Just another testing text.
        </Text>
      </View>
    </View>
  );
}
```

Exploring Layouts and Flexbox

To create our goal manager app, perform the following:

1. In the first nested View, add a TextInput component (don't forget to import!) with a placeholder containing the string 'Your Course Goal'.

```
<View style={styles.appContainer}>
  {/* First section: Input + Button */}
  <View>
    <TextInput placeholder="Your Course Goal" />
```

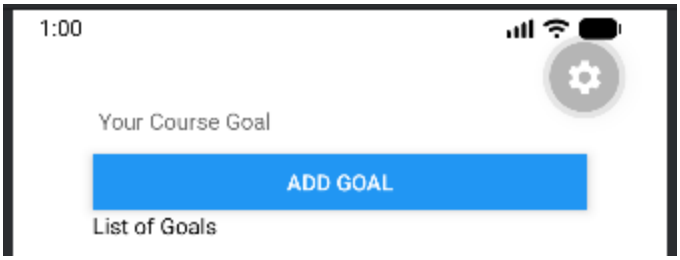
2. Next, add a Button below it with the title 'Add Goal'.

```
<Button title="Add Goal" />
```

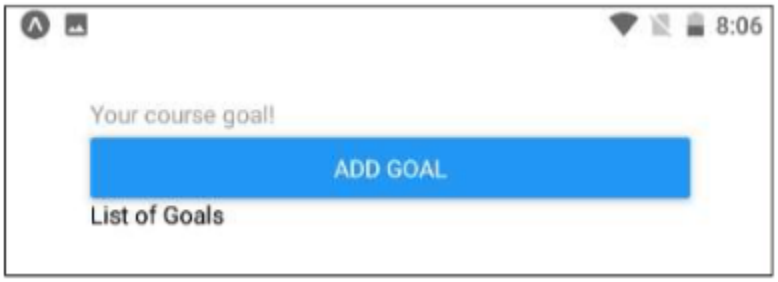
3. In the second nested View, add a Text component that says "List of Goals". This will be a goal later on.

```
{/* Second section: List of Goals */}
<View>
  <Text>List of Goals</Text>
</View>
```

4. Show the output.



5. Create a styles object called appContainer meant to contain the entire app with a padding of 50 pixels. Your output must be similar to the output below.




```javascript
import { StyleSheet, Text, View, Button, TextInput } from 'react-native';

export default function App() {
  return (
    <View style={styles.appContainer}>
      {/* First section: Input + Button */}
      <View>
        <TextInput placeholder="Your Course Goal" />
        <Button title="Add Goal" />
      </View>

      {/* Second section: List of Goals */}
      <View>
        <Text>List of Goals</Text>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  appContainer: {
    padding: 50,
  },
});
```

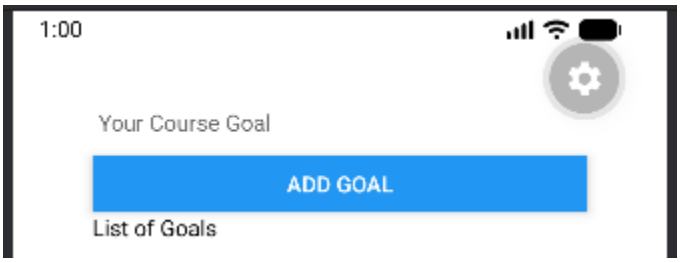
Now, we will use FlexBox

1. Our goal is to make the button and the textinput components be next to each other. To do this, we are

going to create a new styles object for the view that holds both components-we will call it inputContainer. Add flexDirection set to row and justifyContent set to space-between. Assign it to that View component.

```
inputContainer: {
  flexDirection: 'row',
  justifyContent: 'space-between',
},
```

2. Show your output from #1.



3. Next, we want to distinguish the textInput component. Create a new object under styles called textinput, add a border with a width of 2 and a color of #cccccc. Assign to the TextInput then show the output.

```
textInput: {
  borderWidth: 2,
  borderColor: '#cccccc',
  width: '80%',
  marginRight: 8,
  padding: 3,
},
```

- FOR THE OUTPUT SEE NUMBER 5

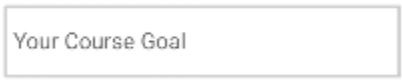
4. Next, add a width of 80%. Make sure it is a string. Show the output.

```
textInput: {
  borderWidth: 2,
  borderColor: '#cccccc',
  width: '80%',
  marginRight: 8,
  padding: 3,
},
```

- FOR THE OUTPUT SEE NUMBER 5

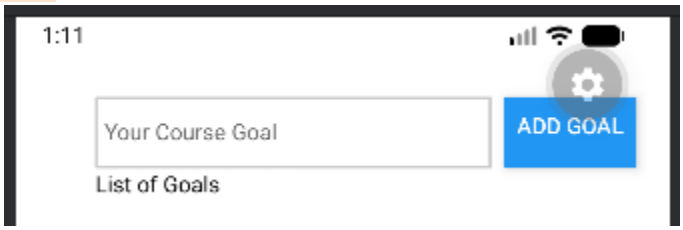
5. Add a marginRight with a value 8 and a padding value 3. Show the output.

```
textInput: {
  borderWidth: 2,
  borderColor: '#cccccc',
  width: '80%',
  marginRight: 8,
  padding: 3,
},
```



6. Your output must be similar to the figure below.

- MY OUTPUT



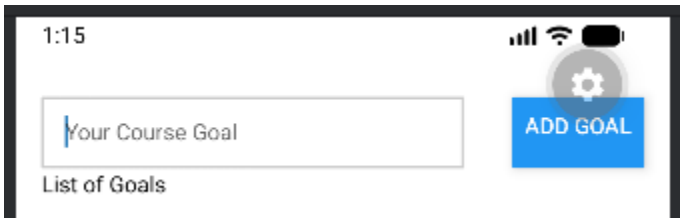

```javascript
import { StyleSheet, Text, View, Button, TextInput } from 'react-native';

export default function App() {
  return (
    <View style={styles.appContainer}>
      {/* Input + Button side by side */}
      <View style={styles.inputContainer}>
        <TextInput style={styles.textInput} placeholder="Your Course Goal" />
        <Button title="Add Goal" />
      </View>

      {/* List of Goals */}
      <View>
        <Text>List of Goals</Text>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  appContainer: {
    padding: 50,
  },
  inputContainer: {
    flexDirection: 'row',
    justifyContent: 'space-between',
  },
  textInput: {
    borderWidth: 2,
    borderColor: '#cccccc',
    width: '80%',
    marginRight: 8,
    padding: 3,
  },
});
```

Improving the Layout

1. Change paddings in appContainer to padding Top: 50, padding Horizontal: 16 and adding textInput width to 70% and padding to 13. Show the output so far.

```
appContainer: {
  flex: 1,
  paddingTop: 50,
  paddingHorizontal: 16,
},
```

```
textInput: {
  borderWidth: 2,
  borderColor: '#cccccc',
  width: '70%',
  marginRight: 8,
  padding: 13,
},
```

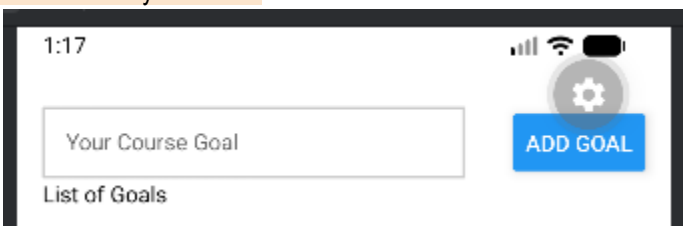


2. Notice the weird placement of the text on your button now. Try to add a style to prop to fix this. What happened?

- When I added styles, the button text looked misaligned. This happens because the button is inside a flex row and inherits spacing.

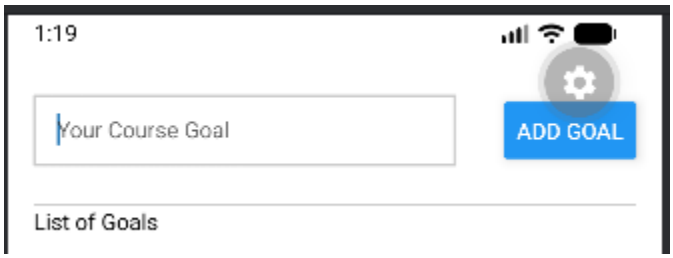
3. Add alignitems: center to the inputContainer. Did this fix the problem? What does this do?

- Button text is vertically centered.



4. To further fix the layout, add a paddingBottom to the InputContainer with a value of 24, create a 1 pixel border on the bottom with the same color as the textInput's border. Show the result.

```
inputContainer: {
  flexDirection: 'row',
  justifyContent: 'space-between',
  alignItems: 'center',
  paddingBottom: 24,
  borderBottomWidth: 1,
  borderBottomColor: '#cccccc',
},
```



5. The textInput area has to take ¼ of the overall available height. To do this, a flex property has to be added. Add flex 1 to the inputContainer.

- Input area now takes ¼ of the screen height.

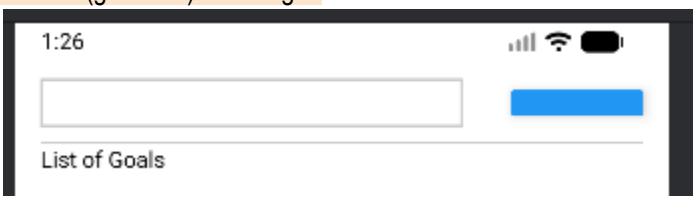
```
inputContainer: {
  flexDirection: 'row',
  justifyContent: 'space-between',
  alignItems: 'center',
  paddingBottom: 24,
  borderBottomWidth: 1,
  borderBottomColor: '#cccccc',
  flex: 1
},
```

6. We are also going to add a new object under styles for the second nested view to support this. Create a new object called goalsContainer and give it a flex value of 3.

```
goalsContainer: {
  flex: 3,
},
```

7. Show the output so far. Is that the expected output?

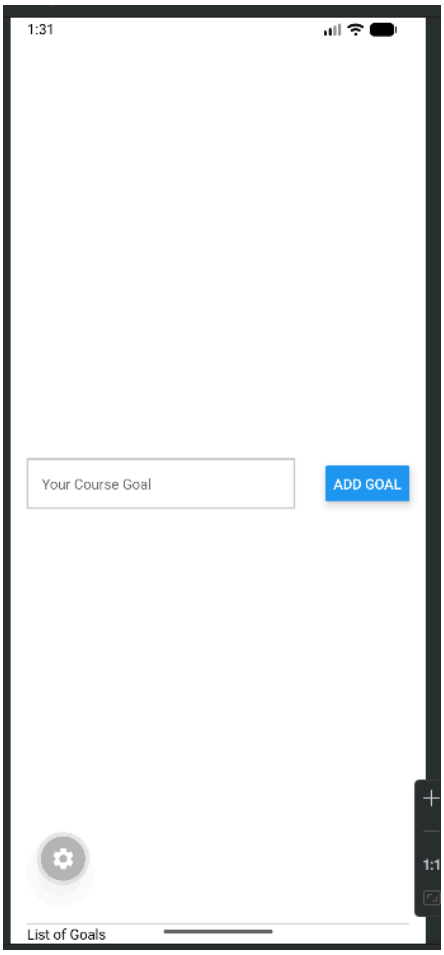
- Top section (input + button) = ¼ height.
- Bottom section (goals list) = ¾ height.



8. Working on the inner containers isn't enough. The outer container must also be adjusted. The appContainer must be forced to take all the available height. Add flex: 1. Show the output.

- Outer container fills the entire screen height.

```
appContainer: {
  paddingTop: 50,
  paddingHorizontal: 16,
  flex: 1
},
```



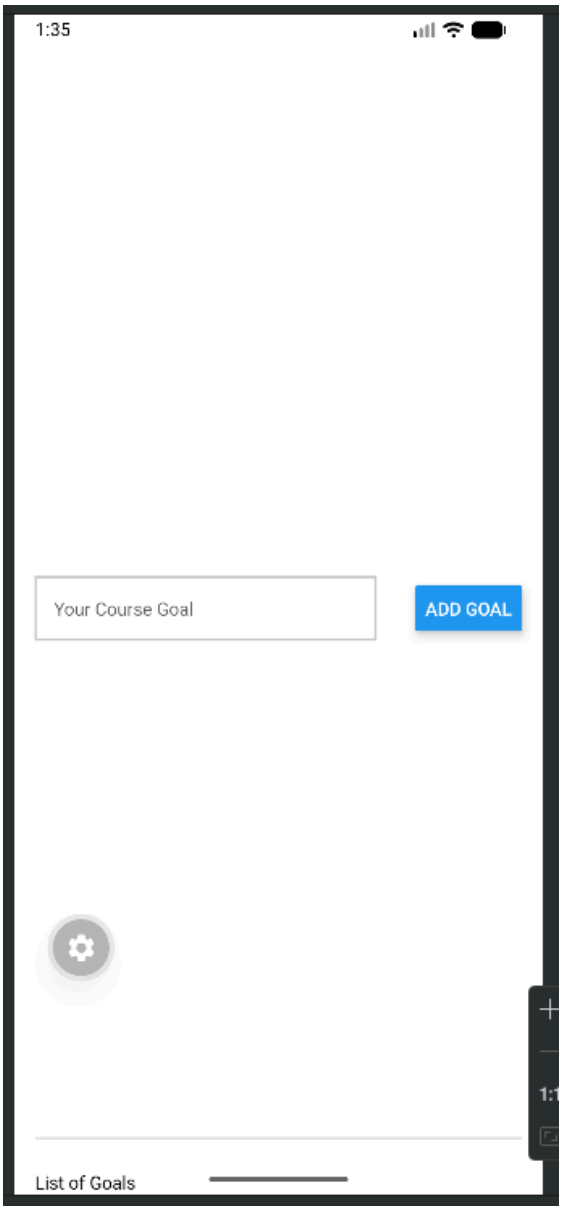
9. Change the flex in goalsContainer to 4. Change paddingBottom to marginBottom in the inputContainer. Show the output.

```
goalsContainer: {
  flex: 4,
},
```

```
inputContainer: {
  flexDirection: 'row',
  justifyContent: 'space-between',
  alignItems: 'center',
  marginBottom: 24,
  borderBottomWidth: 1,
  borderBottomColor: '#cccccc',
  flex: 1
},
```

10. Change the flex in goalsContainer to 5. Show the output. This will be your final layout for now as we proceed with the next steps.

```
goalsContainer: {
  flex: 5,
},
```



JavaScript

```
import { StyleSheet, Text, View, Button, TextInput } from 'react-native';

export default function App() {
  return (
    <View style={styles.appContainer}>
      {/* Input + Button side by side */}
      <View style={styles.inputContainer}>
```

```
        <TextInput style={styles.textInput} placeholder="Your Course Goal" />
        <Button title="Add Goal" />
      </View>

      {/* List of Goals */}
      <View>
        <Text>List of Goals</Text>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  appContainer: {
    paddingTop: 50,
    paddingHorizontal: 16,
    flex: 1
  },
  inputContainer: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginBottom: 24,
    borderBottomWidth: 1,
    borderBottomColor: '#cccccc',
    flex: 1
  },
  textInput: {
    borderWidth: 2,
    borderColor: '#cccccc',
    width: '70%',
    marginRight: 8,
    padding: 13,
  },
  goalsContainer: {
    flex: 5,
  },
});
```

Handling Events

Now, we have a textbox for user input and a button that should place this text into the goal list given when the button is tapped. In order to make these happen, we have to handle events.

1. Make two functions and add them before the return statement in your export default function App():

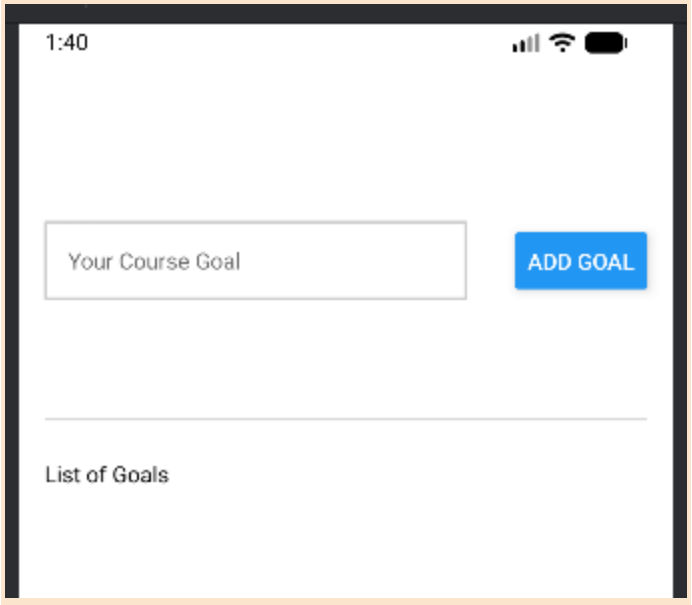
   function goalInputHandler() ();
   function addGoalHandler() {};

It must now look similar to the figure below.

```
export default function App() {
  function goalInputHandler() {};

  function addGoalHandler() {};

  return (
```

- MY OUTPUT

```javascript
export default function App() {
  // Handles text input changes
  function goalInputHandler() {}

  // Handles adding a new goal
  function addGoalHandler() {}

  return (
    <View style={styles.appContainer}>
      <View style={styles.inputContainer}>
        <TextInput
          style={styles.textInput}
          placeholder="Your Course Goal"
          onChangeText={goalInputHandler}
        />
        <Button title="Add Goal" onPress={addGoalHandler} />
      </View>

      <View style={styles.goalsContainer}>
        <Text>List of Goals</Text>
      </View>
    </View>
  );
}
```



2. First, we will connect the textinput component that should take in the texts given by the user through the use of the onChange Text={} prop built-in with React Native. The onChange Text prop wants a function as a value; a pointer at a function. So, we'll pass the goalInputHandler. But not as a function call.

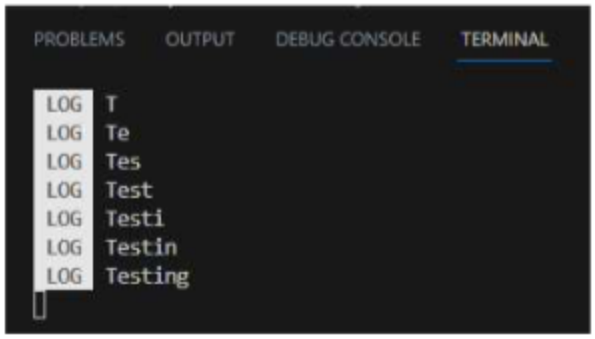
This now means that the goallinputHandler function receives the entered value automatically. The entered value becomes the parameter to the function, so we add a parameter to this function called entered Text.

```
    <TextInput
      style={styles.textInput}
      placeholder="Your Course Goal"
      onChangeText={goalInputHandler}
    />
    <Button title="Add Goal" onPress={addGoalHandler} />
  </View>

  <View style={styles.goalsContainer}>
    <Text>List of Goals</Text>
  </View>
</View>
```

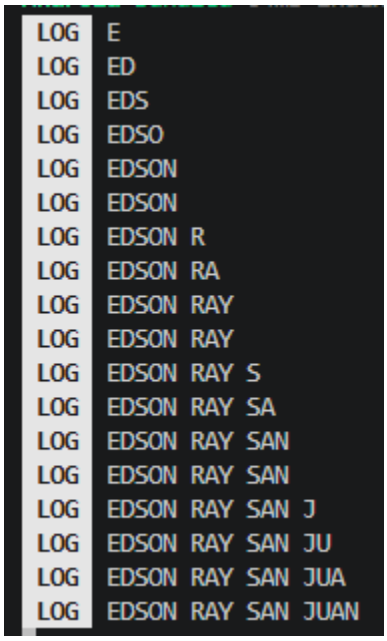
To check, add console.log(entered Text); to the body of the function. Save and then type "Testing" to see whether the connection works properly.

```
function goalInputHandler(enteredText) {
  console.log(enteredText);
}
```

Your output should be similar to the figure shown below.



- MY OUTPUT



JavaScript

```javascript
import { StyleSheet, Text, View, Button, TextInput } from 'react-native';

export default function App() {
  function goalInputHandler(enteredText) {
    console.log(enteredText);
  }

  function addGoalHandler() {}

  return (
    <View style={styles.appContainer}>
      <View style={styles.inputContainer}>
        <TextInput
          style={styles.textInput}
          placeholder="Your Course Goal"
          onChangeText={goalInputHandler}
        />
```

```
        <Button title="Add Goal" onPress={addGoalHandler} />
      </View>

      <View style={styles.goalsContainer}>
        <Text>List of Goals</Text>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  appContainer: {
    paddingTop: 50,
    paddingHorizontal: 16,
    flex: 1
  },
  inputContainer: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginBottom: 24,
    borderBottomWidth: 1,
    borderBottomColor: '#cccccc',
    flex: 1
  },
  textInput: {
    borderWidth: 2,
    borderColor: '#cccccc',
    width: '70%',
    marginRight: 8,
    padding: 13,
  },
  goalsContainer: {
    flex: 5,
  },
});
```

3. However, this saved text should be handled by the button, not the text box. So, now that we've confirmed that the value per keystroke is passed, remove the contents of goalInputHandler's function body.

   In React Native, we have to use the onPress={ prop to connect our function passed as input to the prop. It should now appear as onPress={addGoalHandler};, and similar to earlier, it must not be in the form of a function call.

   - DONE

4. Now that it's connected, we want to get the enteredText in the textInput to be handled by our button. We can do this through states. At the top of your imports, include:

   import {useState} from 'react';

```
import { useState } from 'react';
```

To use the state, first call useState and set as an empty string initially.

const[]= useState('');

```
const [enteredGoalText, setEnteredGoalText] = useState('');
```

- Every keystroke updates the state instead of just logging.

The idea is we want to include within the the user input state or enteredGoalText state which can be updated by the setEnteredGoalText function. This is standard React syntax.

```
function goalInputHandler(enteredText) {
  setEnteredGoalText(enteredText);
}
```

- Every keystroke updates the state instead of just logging.

```
function addGoalHandler() {
    console.log(enteredGoalText);
}
```

- When the button is pressed, it uses the saved state value (not the keystrokes directly).

5. Going back to goalInputHandler, we want to call the update function from the useState call above and pass the entered Text as its parameter. Your code should look similar to the figure below.

```
export default function App() {
  const [enteredGoalText, setEnteredGoalText] = useState('');

  function goalInputHandler(enteredText) {
    setEnteredGoalText(enteredText);
  };

  function addGoalHandler() {};
```

- DONE

6. We now want to test if we have access to this text from the useState. Add console.log(enteredGoalText); in your addGoalHandler's function body. Save and type down "Testing and your output should be similar to the figure below.

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL

LOG  Testing
```

- MY OUTPUT

```
LOG  Testing
LOG  Edson Ray San Juan
```

By the end of step 6, your code should be similar to the figure below.

```javascript
JS App.js > ...
import { useState } from 'react';
import { StyleSheet, Text, View, Button, TextInput } from 'react-native';

export default function App() {
  const [enteredGoalText, setEnteredGoalText] = useState('');

  function goalInputHandler(enteredText) {
    setEnteredGoalText(enteredText);
  };

  function addGoalHandler() {
    console.log(enteredGoalText);
  };
```

- MY CODE

```javascript
import { useState } from 'react';
import { StyleSheet, Text, View, Button, TextInput } from 'react-native';


export default function App() {
  const [enteredGoalText, setEnteredGoalText] = useState('');
  function goalInputHandler(enteredText) {
    setEnteredGoalText(enteredText);
  }

  function addGoalHandler() {
      console.log(enteredGoalText);
  }

  return (
    <View style={styles.appContainer}>
      <View style={styles.inputContainer}>
        <TextInput
          style={styles.textInput}
          placeholder="Your Course Goal"
          onChangeText={goalInputHandler}
        />
        <Button title="Add Goal" onPress={addGoalHandler} />
      </View>

      <View style={styles.goalsContainer}>
        <Text>List of Goals</Text>
      </View>
    </View>
  );
}
```

```
const styles = StyleSheet.create({
  appContainer: {
    paddingTop: 50,
    paddingHorizontal: 16,
    flex: 1
  },
  inputContainer: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginBottom: 24,
    borderBottomWidth: 1,
    borderBottomColor: '#cccccc',
    flex: 1
  },
  textInput: {
    borderWidth: 2,
    borderColor: '#cccccc',
    width: '70%',
    marginRight: 8,
    padding: 13,
  },
  goalsContainer: {
    flex: 5,
  },
});
```

Managing the List of Course Goals

Now, we want to be able to update our list of goals that found in the secondary view component.

1. Typically, when data changes dynamically we use state. Such as in this case, with the data for the list of goals changing, we want to useState as well. Add another useState below the first one created in the earlier steps. However, this second state will be initialized with an empty array instead.

   This state should be named courseGoals that can be changed by the setCourse Goals function.

```
const [courseGoals, setCourseGoals] = useState([]);
```

2. In the body of your addGoalHandler, instead of using the console, use the setCourseGoals. We want to be able to take your existing goals and append (add to the end) a new one.

   We do this by passing an array to the setCourseGoals function.
   setCourse Goals([]);

   Then we use the spread operator to spread existing course goals into the new array, keeping all the existing goals.

   setCourse Goals([...courseGoals]);

And then adding the new goal.

setCourse Goals([...courseGoals, enteredGoal Text]);

HOWEVER, this is also not the best way to update a state if the new state depends on the previous state. Instead of the aforementioned code, we will pass a function to the state updating function that will be automatically called by react.

We will use an arrow function instead.

setCourse Goals((currentCourseGoals) => [...currentCourseGoals, enteredGoal Text,]);

```
function addGoalHandler() {
  setCourseGoals((currentCourseGoals) => [
    ...currentCourseGoals,
    enteredGoalText,
  ]);
}
```

3. At this point, your code should look similar to the figure below.

```
JS App.js > App
import { useState } from 'react';
import { StyleSheet, Text, View, Button, TextInput } from 'react-native';

export default function App() {
  const [enteredGoalText, setEnteredGoalText] = useState('');
  const [courseGoals, setCourseGoals] = useState([]);

  function goalInputHandler(enteredText) {
    setEnteredGoalText(enteredText);
  };

  function addGoalHandler() {
    setCourseGoals((currentCourseGoals) =>[
      ...currentCourseGoals,
      enteredGoalText,
    ]);
  };

  return (
```

- MY OUTPUT

```javascript
import { useState } from 'react';
import { StyleSheet, Text, View, Button, TextInput } from 'react-native';


export default function App() {
```

```
const [enteredGoalText, setEnteredGoalText] = useState('');
const [courseGoals, setCourseGoals] = useState([]);
function goalInputHandler(enteredText) {
  setEnteredGoalText(enteredText);
}

function addGoalHandler() {
  setCourseGoals((currentCourseGoals) => [
    ...currentCourseGoals,
    enteredGoalText,
  ]);
}
```

4. The next goal is to output the list of goals. Since this data is dynamic, we can now refer to courseGoals enclosed in curly brackets then call the map method on it.

{currentGoals.map()}

We will pass here another arrow function, which takes individual courseGoals for the parameter and as a return, it returns a JSX element.

(currentGoals.map((goal) => <Text>{goal}</Text>)}

At this point, your code should look like the figure below.

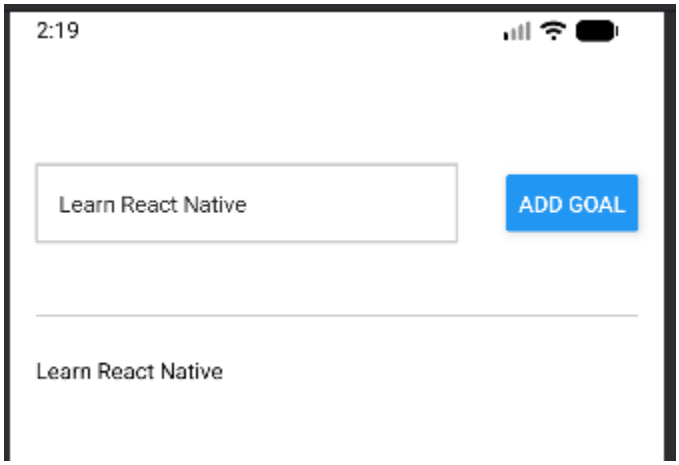
```
return (
  <View style={styles.appContainer}>
    <View style={styles.inputContainer}>
      <TextInput
      placeholder='Your course goal!'
      style={styles.textInput}
      onChangeText={goalInputHandler}
      />
      <Button title='Add Goal' onPress={addGoalHandler} />
    </View>
    <View style={styles.goalsContainer}>
      {courseGoals.map((goal) => <Text>{goal}</Text>)}
    </View>
  </View>
);
}
```

- MY OUTPUT

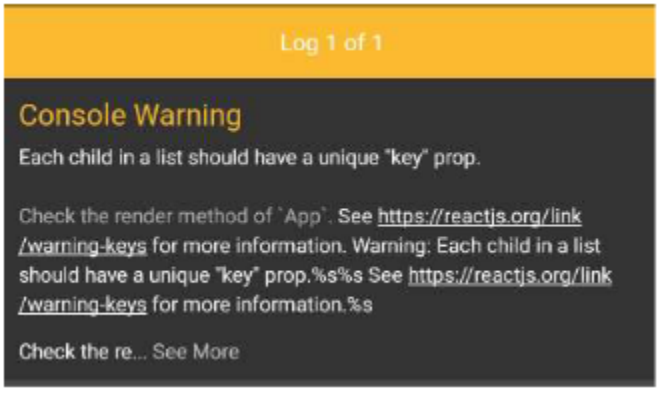
```
<View style={styles.goalsContainer}>
  {courseGoals.map((goal) => (
    <Text key={goal}>{goal}</Text>
  ))}
</View>
```

5. Save and run your code, Input in the textInput area as a goal "Leam React Native". Press on "Add Goal" button. What happens? Was it added? Did anything else happen? Show output/screenshots.

- **What happens?**
  - It outputted what I typed
- **Was it added?**
  - Yes
- **Did anything else happen?**
  - No



You should receive a warning. Not because of React Native, but due to the nature of React in general. The error states that when outputting a list of data every item should receive a key prop.



To make it quick, this is meant for React to manage the list in a more efficient way. Always add the key prop when dealing with individual items being outputted as part of a list.

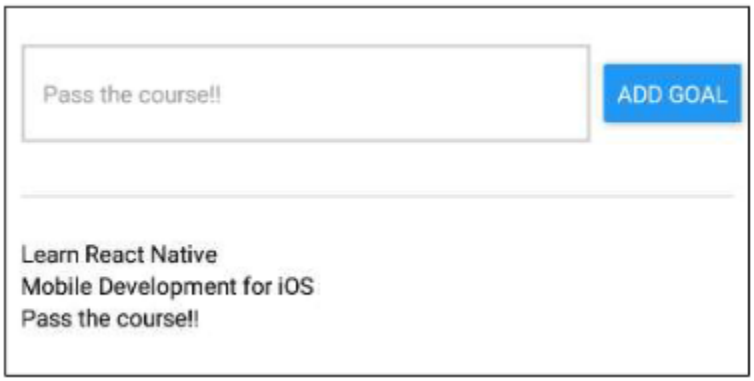
6. The value passed to the key **can be anything** but it must be unique. It has to uniquely identify the concrete value that's being outputted. In this case, each string is assumed to be unique so we'll use the goal string itself.

<Text key={goal}>{goal}</Text>

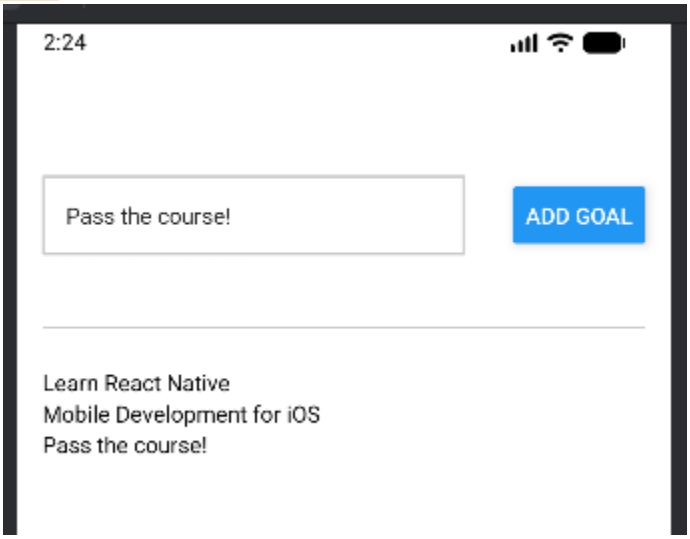
Temporarily, it will work. Test it with different goals and same goals. Such as:

"Learn React Native" / "Mobile Development for iOS" / "Pass the Course" (Unless you do not want to learn anything and not pass the course.)

The app in your simulator should look similar to the figure below.



- MY OUTPUT



JavaScript

```javascript
import { useState } from 'react';
import { StyleSheet, Text, View, Button, TextInput } from 'react-native';


export default function App() {
  const [enteredGoalText, setEnteredGoalText] = useState('');
  const [courseGoals, setCourseGoals] = useState([]);
  function goalInputHandler(enteredText) {
```

```
    setEnteredGoalText(enteredText);
  }

  function addGoalHandler() {
    setCourseGoals((currentCourseGoals) => [
      ...currentCourseGoals,
      enteredGoalText,
    ]);
  }


  return (
    <View style={styles.appContainer}>
      <View style={styles.inputContainer}>
        <TextInput
          style={styles.textInput}
          placeholder="Your Course Goal"
          onChangeText={goalInputHandler}
        />
        <Button title="Add Goal" onPress={addGoalHandler} />
      </View>

      <View style={styles.goalsContainer}>
        {courseGoals.map((goal) => (
          <Text key={goal}>{goal}</Text>
        ))}
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  appContainer: {
    paddingTop: 50,
    paddingHorizontal: 16,
    flex: 1
  },
  inputContainer: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginBottom: 24,
    borderBottomWidth: 1,
    borderBottomColor: '#cccccc',
    flex: 1
  },
  textInput: {
    borderWidth: 2,
    borderColor: '#cccccc',
    width: '70%',
```

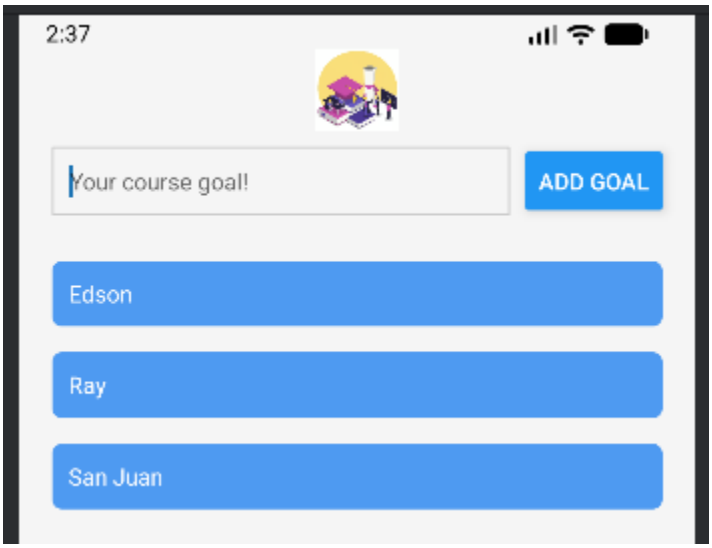
```
    marginRight: 8,
    padding: 13,
  },
  goalsContainer: {
    flex: 5,
  },
});
```

7. Supplementary Activity

Now that you have experienced how to use React Native in the development of a basic application with the guides provided in this material, it is time for you to experiment on your own.

ILO1: Utilize React Native components and build user interfaces and ILO2: Style React Native applications.

- Using the base application created in the procedure of this activity, stylize the application further. You may change fonts, add images, and even further try out different ways to use flexbox. However, one thing must remain: the application must still retain the same function, by the end of this activity, it must still be a goal list application for this course. You're only changing the style.
- Hint: You get points for making aesthetic choices. Please avoid bright, obnoxious and uncoordinated color coordinations. You will be graded accordingly.

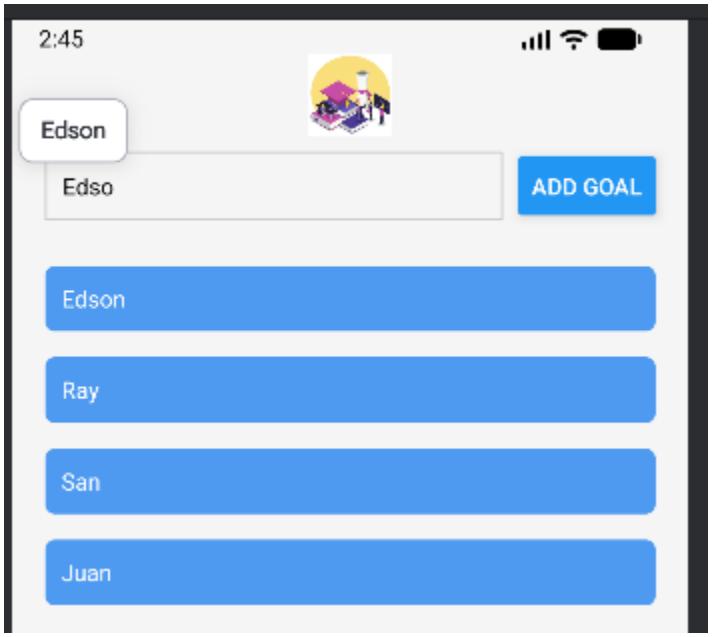


- **OBSERVATION:** I enhanced the design adding extra components such as an Image for a logo and a ScrollView for the goals list, which made the user interface more structured and visually appealing while still retaining its original function as a goal-tracking application.

ILO3: Implement additional interactivity and manage states.

- The application lacks in providing ample user experience. To change this, make it so that every time the "add goal" button is pressed, the previous inputs are also cleared reverting back to the placeholder.

```
function addGoalHandler() {
  setCourseGoals((currentCourseGoals) => [
    ...currentCourseGoals,
    enteredGoalText,
  ]);
  setEnteredGoalText('');
}
```



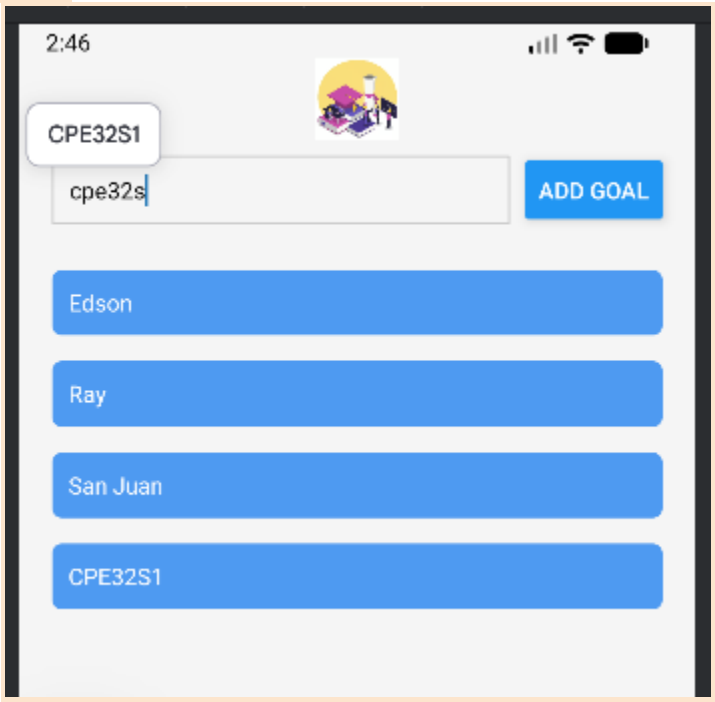
- Sample output for ILO3 in figures below.

Your course goal!
ADD GOAL
Learn React Native

Your course goal!
ADD GOAL
Learn React Native
Mobile App Development

Your course goal!
ADD GOAL
Learn React Native
Mobile App Development
Pass the course!!!!!

- MY OUTPUT


2:46
CPE32S1
cpe32s
ADD GOAL
Edson
Ray
San Juan
CPE32S1

JavaScript

```javascript
import { useState } from 'react';
import { StyleSheet, Text, View, Button, TextInput, ScrollView, Image } from
'react-native';

export default function App() {
  const [enteredGoalText, setEnteredGoalText] = useState('');
  const [courseGoals, setCourseGoals] = useState([]);

  function goalInputHandler(enteredText) {
    setEnteredGoalText(enteredText);
  }

  function addGoalHandler() {
    setCourseGoals((currentCourseGoals) => [
      ...currentCourseGoals,
      enteredGoalText,
    ]);
    setEnteredGoalText('');
  }

  return (
    <View style={styles.appContainer}>
      <Image
        source={{ uri:
'https://www.softo.org/imgs/blog/2023/07/1690190032univer-png.png' }}
        style={styles.logo}
      />
      <View style={styles.inputContainer}>
        <TextInput
          placeholder="Your course goal!"
          style={styles.textInput}
          onChangeText={goalInputHandler}
          value={enteredGoalText}
        />
        <Button title="ADD GOAL" onPress={addGoalHandler} />
      </View>
      <ScrollView style={styles.goalsContainer}>
        {courseGoals.map((goal) => (
          <Text key={goal} style={styles.goalItem}>{goal}</Text>
        ))}
      </ScrollView>
    </View>
  );
}

const styles = StyleSheet.create({
  appContainer: {
    flex: 1,
    padding: 20,
    backgroundColor: '#f5f5f5',
```

```
  },
  logo: {
    width: 50,
    height: 50,
    marginBottom: 10,
    alignSelf: 'center',
  },
  inputContainer: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginBottom: 20,
  },
  textInput: {
    borderWidth: 1,
    borderColor: '#cccccc',
    padding: 10,
    flex: 1,
    marginRight: 10,
    fontFamily: 'Arial',
  },
  goalsContainer: {
    flex: 4,
  },
  goalItem: {
    marginVertical: 8,
    padding: 10,
    backgroundColor: '#4e9af1',
    color: 'white',
    borderRadius: 6,
  },
});
```

8. Summary of the Activity, Conclusion & Lessons Learned from the Module

1. **Summary of the Activity**

In this activity, I created a basic React Native application that demonstrates the use of core components, styling, layout design, and event handling. I built a simple Course Goals app where users can input and add goals dynamically. I applied styling using StyleSheet, organized layouts using Flexbox, and implemented functionality using state management and event handling.

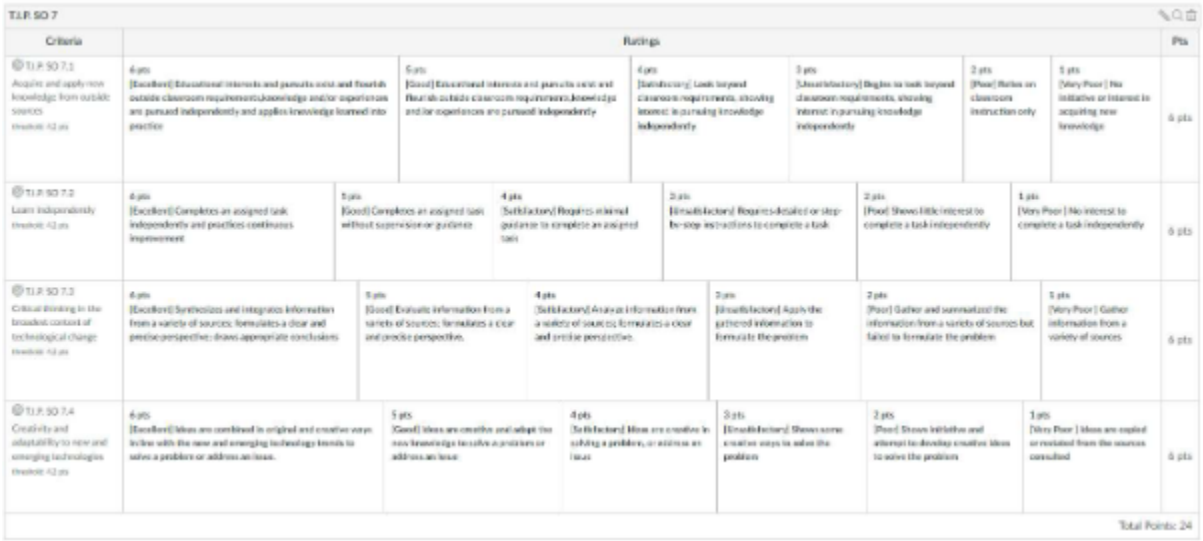
2. **Conclusion**

In conclusion, I built a simple but functional Course Goals app in React Native where I applied key concepts like handling user input, managing state with useState, and dynamically rendering lists. I learned update state, append data using the spread operator, and use unique keys for efficient rendering. In the supplementary activity, I was task to improve the app by enhancing its interface with additional components like an image and a scrollable list, and by adding interactivity such as clearing the input field after each entry. Overall, these improvements made the app more organized, responsive, and user-friendly while deepening my understanding of the core React Native concepts.

3. Lessons Learned from the Module

From this module, I learned how to use core React Native components, apply styles efficiently, and manage layouts using Flexbox. I also gained a better understanding of handling user input, updating state properly, and rendering dynamic data using lists with unique keys. Overall, this module helped me build a strong foundation in developing interactive and responsive mobile applications.

9. Assessment Rubric

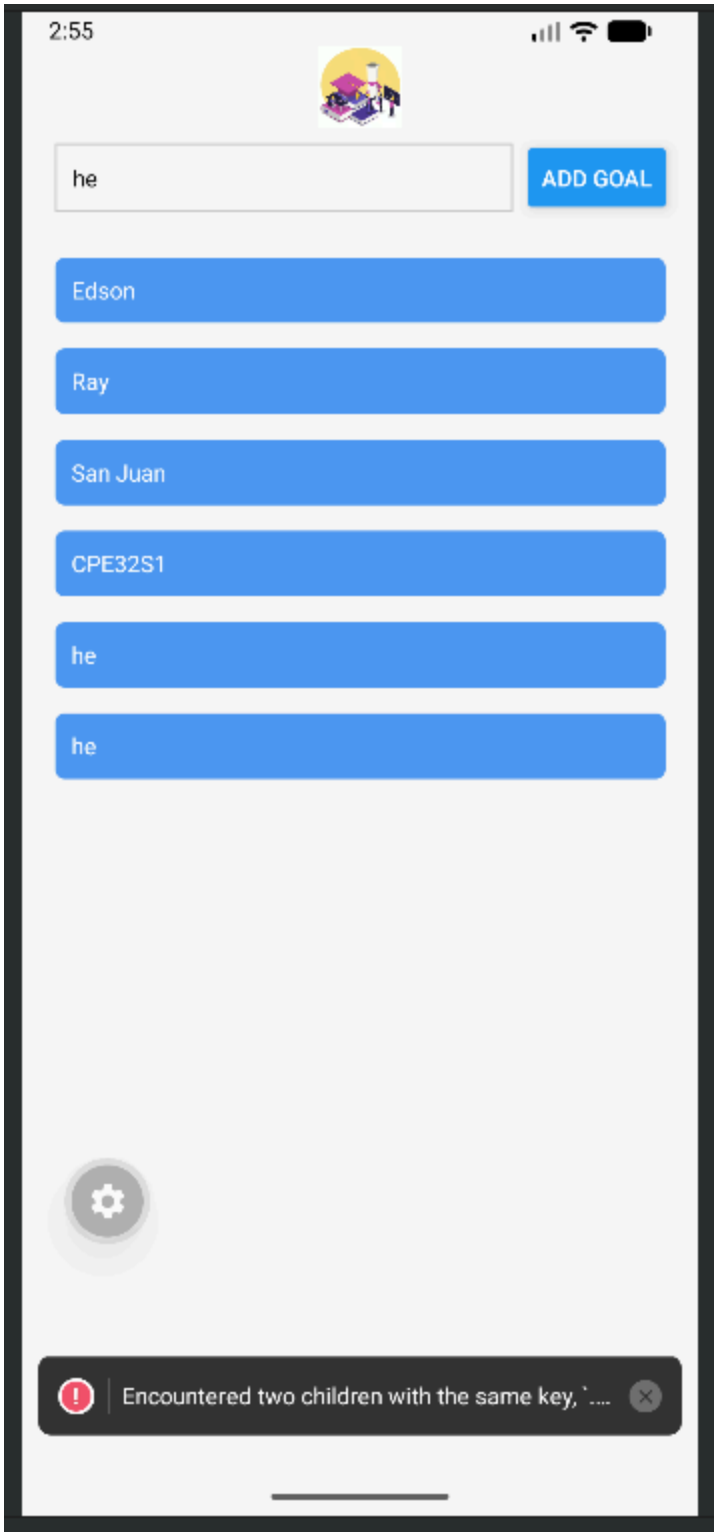


T.I.P. SO 7

Criteria	Ratings						Pts
T.I.P. SO 7.1 Acquire and apply new knowledge from outside sources Criterion 42 pts	6 pts [Excellent] Educational interests and pursuits exist and flourish outside classroom requirements,knowledge and/or experiences are pursued independently and applies knowledge learned into practice	5 pts [Good] Educational interests and pursuits exist and flourish outside classroom requirements,knowledge and/or experiences are pursued independently	4 pts [Satisfactory] Look beyond classroom requirements, showing interest in pursuing knowledge independently	3 pts [Unsatisfactory] Begins to look beyond classroom requirements, showing interest in pursuing knowledge independently	2 pts [Poor] Relies on classroom instruction only	1 pts [Very Poor] No initiative or interest in acquiring new knowledge	6 pts
T.I.P. SO 7.2 Learn independently Criterion 42 pts	6 pts [Excellent] Completes an assigned task independently and practices continuous improvement	5 pts [Good] Completes an assigned task without supervision or guidance	4 pts [Satisfactory] Requires minimal guidance to complete an assigned task	3 pts [Unsatisfactory] Requires detailed or step-by-step instructions to complete a task	2 pts [Poor] Shows little interest to complete a task independently	1 pts [Very Poor] No interest to complete a task independently	6 pts
T.I.P. SO 7.3 Critical thinking in the broadest context of technological change Criterion 42 pts	6 pts [Excellent] Synthesizes and integrates information from a variety of sources; formulates a clear and precise perspective; draws appropriate conclusions	5 pts [Good] Evaluate information from a variety of sources; formulates a clear and precise perspective.	4 pts [Satisfactory] Analyze information from a variety of sources; formulates a clear and precise perspective.	3 pts [Unsatisfactory] Apply the gathered information to formulate the problem	2 pts [Poor] Gather and summarized the information from a variety of sources but failed to formulate the problem	1 pts [Very Poor] Gather information from a variety of sources	6 pts
T.I.P. SO 7.4 Creativity and adaptability to new and emerging technologies Criterion 42 pts	6 pts [Excellent] Ideas are combined in original and creative ways in line with the new and emerging technology trends to solve a problem or address an issue.	5 pts [Good] Ideas are creative and adapt the new knowledge to solve a problem or address an issue	4 pts [Satisfactory] Ideas are creative in solving a problem, or address an issue	3 pts [Unsatisfactory] Shows some creative ways to solve the problem	2 pts [Poor] Shows initiative and attempt to develop creative ideas to solve the problem	1 pts [Very Poor] Ideas are copied or revisited from the sources consulted	6 pts

Total Points: 24

10. Additional Screenshots




```
Android Bundled 49ms index.js (1 module)
 ERROR  Encountered two children with the same key, `%s`. Keys should be unique so that
nd/or omitted — the behavior is unsupported and could change in a future version. he
 ERROR  Encountered two children with the same key, `%s`. Keys should be unique so that
nd/or omitted — the behavior is unsupported and could change in a future version. .$he
```

- When I input the word “he” again, as you can see in the output, it shows an error because the word “he” already exists in the list.